

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Namespaces and Assemblies
- ▶ Lambda Expressions
- ▶ Generics

Let's Start

Namespaces and Assemblies

▶ Namespaces

A namespace is a container for classes, interfaces, structs, enums, and delegates, used to organize code logically and avoid naming conflicts.

Key Points:

- **Syntax:** `namespace NamespaceName { // members }`
- Provides a way to group related types (e.g., `System.Collections.Generic`).
- Helps prevent **name clashes** by isolating identifiers.
- Multiple levels of namespaces can be nested.

Namespaces and Assemblies

```
namespace MyApp.Utilities
{
    public class Logger
    {
        public void Log(string message) => Console.WriteLine(message);
    }
}
```

Namespaces and Assemblies

▶ Assemblies

An assembly is a compiled code library used for deployment, versioning, and security in .NET. It contains **Intermediate Language (IL)** code and metadata.

Types of Assemblies:

- **Executable (EXE)**: Entry point for applications.
- **Library (DLL)**: Reusable components.

Key Components:

- **Manifest**: Metadata about the assembly (name, version, culture, etc.).
- **IL Code**: Compiled .NET code.
- **Metadata**: Descriptions of types, members, and references.

Lambda Expressions

▶ Lambda Expressions

A **lambda expression** is a concise way to represent an anonymous method using the => (lambda) operator.

Syntax:

•(parameters) => expression

```
nums.Where(x => x > 5);
```

Generics

- Generic is to allow type to be a parameter to methods, classes, and interfaces.
- It enables to create classes, interfaces, and methods that work with different data types without having to define the data type explicitly.
- It provides type safety. When a generic class, interface or method is defined, the type parameter that it can work with can be specified.
- This ensures that the code will only accept and operate on data types that match the specified type parameter, preventing errors that can occur due to data type mismatches.
- C# provides a number of built-in generic types, such as `List<T>`, `Dictionary<TKey, TValue>`,

Generics

```
class DataStore<T>
{
    private T[] _data = new T[10];
    public void AddOrUpdate(int index, T item) {
        if(index >= 0 && index < 10) _data[index] = item;
    }
    public T GetData(int index) {
        if(index >= 0 && index < 10) return _data[index];
        else return default(T);
    }
}
```

```
DataStore<string> cities = new
DataStore<string>();
cities.AddOrUpdate(0, "Mumbai");
cities.AddOrUpdate(1, "Chicago");
cities.AddOrUpdate(2, "London");

DataStore<int> emplds = new DataStore<int>();
emplds.AddOrUpdate(0, 50);
emplds.AddOrUpdate(1, 65);
emplds.AddOrUpdate(2, 89);
```

Thank you

Self Learning

Delegates and Events

- Creating and using delegates
- Anonymous methods
- Events and event handlers

Advanced C# Concepts

- Boxing and Unboxing
- Type Casting
- Extension Methods